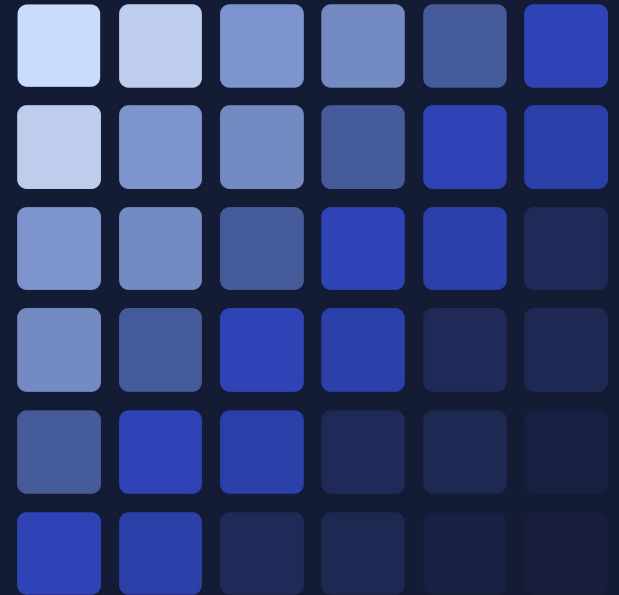


COLLEGE SEMINAR · DEEP LEARNING

Convolutional Neural Networks

How machines learn to see — from raw pixels to understanding

Tushank Patel · July 2026



MOTIVATION

Why do we need CNNs?

An image is just a grid of numbers. A modest 224×224 RGB photo has over 150,000 input values. Feeding these into a fully connected network ignores spatial structure and explodes in parameters.

150K+

input values in one small 224×224 image

~1.2 B

weights if the first dense layer had 8,000 neurons

×1000s

fewer parameters with shared convolution filters

Three ideas CNNs exploit



Local receptive fields

Each neuron looks at a small patch, not the whole image — nearby pixels are related.



Weight sharing

The same filter slides across the image, so one pattern detector works everywhere.



Hierarchy of features

Edges combine into textures, textures into parts, parts into whole objects.

CORE OPERATION

The convolution: a sliding filter

Input 5×5

0	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter 3×3

1	0	-1
1	0	-1
1	0	-1

*

=

Feature map 3×3

-3	0	2
-3	-2	2
-3	-1	2

Element-wise multiply & sum. The highlighted 3×3 patch × the filter gives the first output value. The filter then slides one step (the stride) and repeats — producing a feature map that lights up wherever the pattern (here, a vertical edge) appears.

VOCABULARY

Kernel / filter

the small weight matrix being learned

Stride

how far the filter moves each step

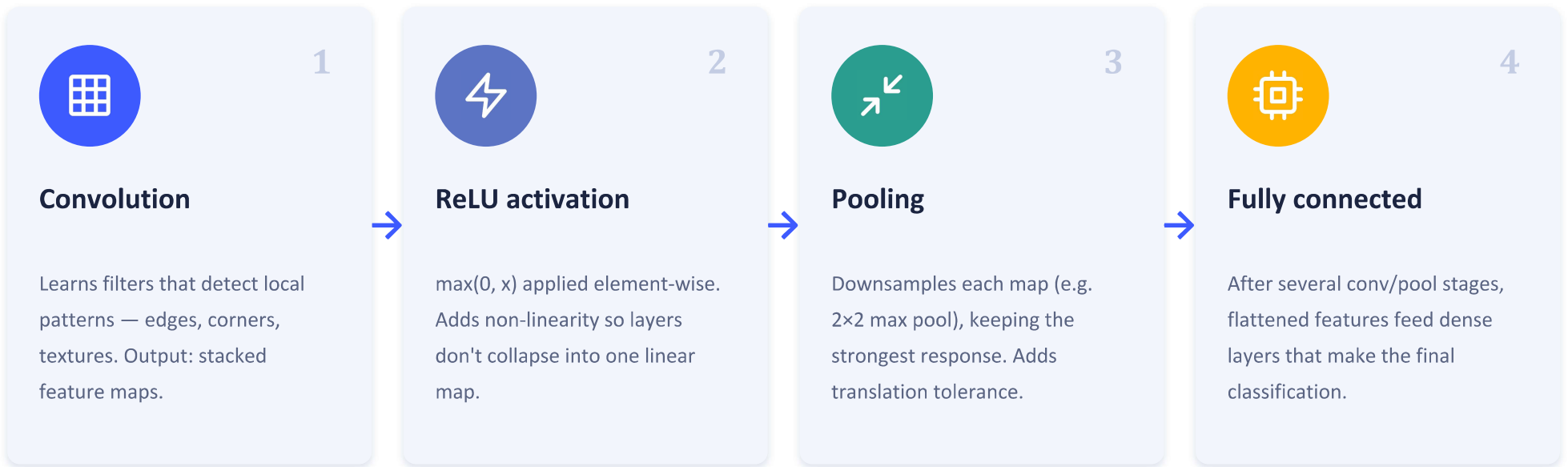
Padding

border zeros that preserve size

Channels

one feature map per filter

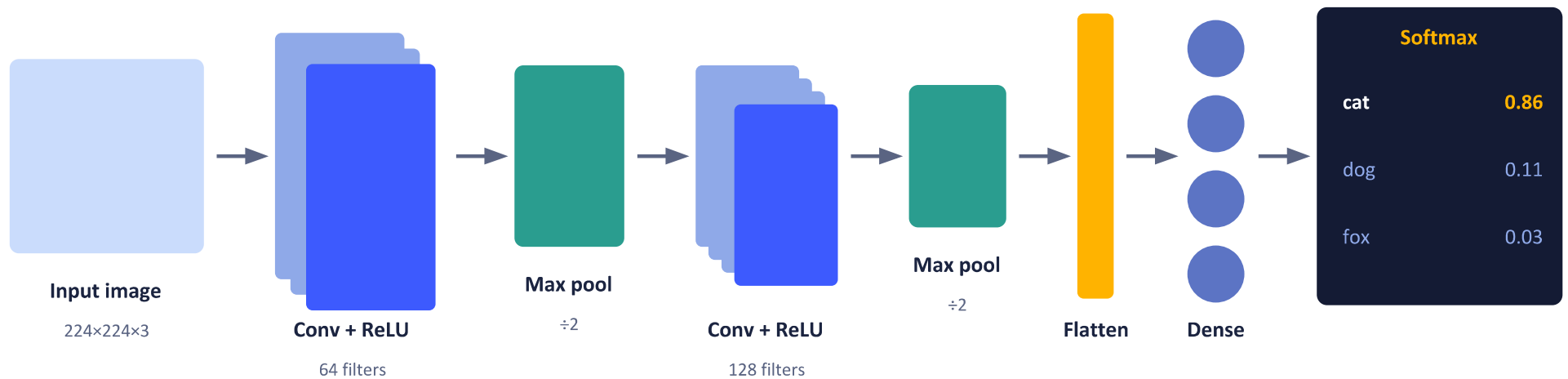
The layers of a CNN



The pattern: [Conv → ReLU → Pool] repeated several times, then Flatten → Dense → Softmax.

END TO END

Anatomy of a CNN classifier



Feature hierarchy: early layers detect edges and colors → middle layers detect textures and parts (eyes, wheels) → late layers respond to whole objects. Spatial size shrinks while the number of channels grows.

A BRIEF HISTORY

Landmark architectures

1998

LeNet-5

Yann LeCun's digit reader for postal codes — proved the conv/pool recipe.

~60K params

2012

AlexNet

Won ImageNet by a landslide (top-5 error 15.3% vs 26.2%) using GPUs, ReLU and dropout. Sparked the deep-learning boom.

60M params

2014

VGG-16

Showed depth matters: 16 layers of simple, uniform 3×3 convolutions.

138M params

2015

ResNet

Skip connections let gradients flow through 100+ layers — surpassed human-level top-5 error on ImageNet.

152 layers

Since ~2020, Vision Transformers rival CNNs on huge datasets — but CNNs remain the efficient default for most vision tasks.

IN THE WILD

Where CNNs are used today



Image classification

Tagging photos, content moderation, species & landmark recognition.



Object detection

YOLO & Faster R-CNN localize and label multiple objects per frame.



Medical imaging

Detecting tumours in X-rays, MRIs and retinal scans at expert-level accuracy.



Autonomous driving

Lane, pedestrian and sign recognition from on-board cameras.



Face recognition

Phone unlock, identity verification, photo grouping.



Video & beyond

Action recognition, style transfer, super-resolution — even audio spectrograms.

Strengths & limitations



Strengths

Parameter efficient — weight sharing means far fewer weights than dense nets

Translation tolerant — a cat is a cat wherever it sits in the frame

Learns its own features — no hand-engineered descriptors like SIFT/HOG

Transfer learning — pretrained backbones adapt to new tasks with little data



Limitations

Data hungry — strong results usually need large labelled datasets

Compute heavy — training deep models demands GPUs and energy

Not rotation/scale invariant — needs augmentation to handle viewpoint change

Opaque & brittle — hard to interpret; fooled by adversarial pixel noise

KEY TAKEAWAYS

What to remember

1

Convolution = learned pattern detectors

Small shared filters slide over the image, exploiting locality and cutting parameters dramatically.

2

Depth builds a feature hierarchy

Edges → textures → parts → objects; resolution shrinks while channels grow.

3

The recipe is modular

[Conv → ReLU → Pool] × N, then dense layers — from LeNet to ResNet, the same core idea.

4

Powerful but not magic

Data- and compute-hungry, opaque, and vulnerable to adversarial noise — use with judgment.

Thank you — questions?

